Australian Wildfire Prediction Using Historical Weather Data

Charles Gallagher

May 25, 2022

Table of Contents

1 Executive Summary **4**
1.1 Purpose 4
1.2 Methods 4
1.3 Results 4

2 Introduction and Motivation **5**

3 Related Work **5**

4 Methods **7**
4.1 VIF Analysis 7
4.2 Train Test Split Without Randomization 7
4.3 Multinomial Logistic Regression 8
4.3.a Hyper-Parameter Tuning 8
4.3.b Testing on the Validation Set 8
4.4 XGBoost 8
4.4.a Hyper-Parameter Tuning 8
4.4.b Testing on the Validation Set 8
4.5 Multilayer Perceptron 9
4.5.a Hyper-Parameter Tuning 9
4.5.b Testing on the Validation Set 9
4.6 K-Nearest Neighbors 10
4.6.a Hyper-Parameter Tuning 10
4.6.b Testing on the Validation Set 10
4.7 Random Forest Classifier 10
4.7.a Hyper-Parameter Tuning 10
4.7.b Testing on the Validation Set 10
4.8 Logistic GAM Classifier 11
4.8.a Hyper-Parameter Tuning 11
4.8.b Testing on the Validation Set 11
4.9 Tree Classifier 11
4.9.a Hyper-Parameter Tuning 11
4.9.b Testing on the Validation Set 11
4.10 Support Vector Classifier 12
4.10.a Hyper-Parameter Tuning 12
4.10.b Testing on the Validation Set 12
4.11 Naive Bayes Classifier 12
4.11.a Hyper-Parameter Tuning 12
4.11.bTesting on the Validation Set 12
4.12 XGBoost Random Forest Ensemble Methods 13
4.12.a Hyper-Parameter Tuning 13

4.12.b Testing on the Validation Set 13
4.13 Experimentation with Bagging and Importance 13
4.14 Experimentation with LogisticGAM 14
4.15 Experimentation with Resampling Methods 14
4.16 Data Table Pivot 14

5 Future Directions 16

This section is for open questions which I still have after completing my analysis. 16

6 Description of Effort 16
6.1 What I Learned vs. What I Knew 16
6.2 Libraries and Packages Used 16
6.3 Allocation of Time 17

7 Appendix 18
Item 1: 18

8 Bibliography 19

1 Executive Summary

1.1 Purpose

The purpose of this paper is to explain how I arrived at a model for predicting wildfires in Australia and to present the results of that model. More broadly, the purpose of this project is to demonstrate the efficacy of predicting wildfires from historical weather data alone, in the hopes that it will inspire others to create better models which can predict forest fires from future weather predictions, perhaps with 5 day lead times. With better modeling tens of thousands of human lives lost, millions of animal lives, and billions in property damage could potentially be stopped.

1.2 Methods

A variety of methods were used for this project. These include a data table pivot, VIF analysis, non-randomized train/test split, multinomial logistic regression (MAXENT), extreme gradient boost classification (XGBoost), multi-layer perceptron classification (MLP), k-nearest neighbors classification (KNN), random forest classification (RF), logistic generalized additive models classification (LogisticGAM), tree based classification (TR), support vector classification (SVC), naive Bayes classification (NB), extreme gradient boosting-random forest ensemble methods (XGBRF), experimentation with bagging and importance, and experimentation with logisticGAM over various time intervals. Data was split into 70%, 20%, and 10% training, validation, and test sets respectively. Because the data are sequential in time, it was necessary to avoid predicting the past with the future. So data were split sequentially so that the ordinal position was preserved across training, validation, and test sets. Further, the 70% training set was split into a 50% and 20% (total) subtraining training and subtraining testing sets. In this way, a "competition" was performed between models. The 50/20 split was used to test hyperparameters. The resultant model was then trained on the 70% training set, tested on the validation set, and compared to all other models. The most successful of these was trained on the training and validation sets concatenated together and tested on the test set. These results are reported in the following section.

1.3 Results

The winning model from our "competition" was the random forest classifier. I report an accuracy score of 0.8521 and AUC-ROC of 0.9277. The following summarizes my other findings:

	precision	recall	f1-score	support
0	0.76	0.83	0.79	1377
1	0.91	0.87	0.89	2667
accuracy			0.85	4044
macro avg	0.83	0.85	0.84	4044
weighted avg	0.86	0.85	0.85	4044

These are a “no-fire” prediction precision of 0.76 and recall of .83 as well as a “fire” prediction precision of .91 and recall of .87. Overall, these findings are just better than our goal of 80% accuracy.

2 Introduction and Motivation

This project is important because wildfires are dangerous. They cause tens of billions of dollars of property damage each year **[2]** and kill approximately 33,000 people annually **[3]**. In Brazil alone, during the year 2020, wildfires killed an estimated 17 million animals and burned up over a third of the world's largest wetland **[4]**. Wildfire prediction is one tool to help prevent tens of billions of dollars of property damage, tens of thousands of lives lost, and devastating damage to ecosystems around the world. This project, while simple and ultimately not that useful for actual wildfire prediction, demonstrates that even simple machine learning models applied to historical weather data have strong predictive power for this problem. At the very least, this project demonstrates that wildfire prediction is not an impossible problem and that with the right predictors and models we can predict wildfire occurrence with even better accuracy and precision.

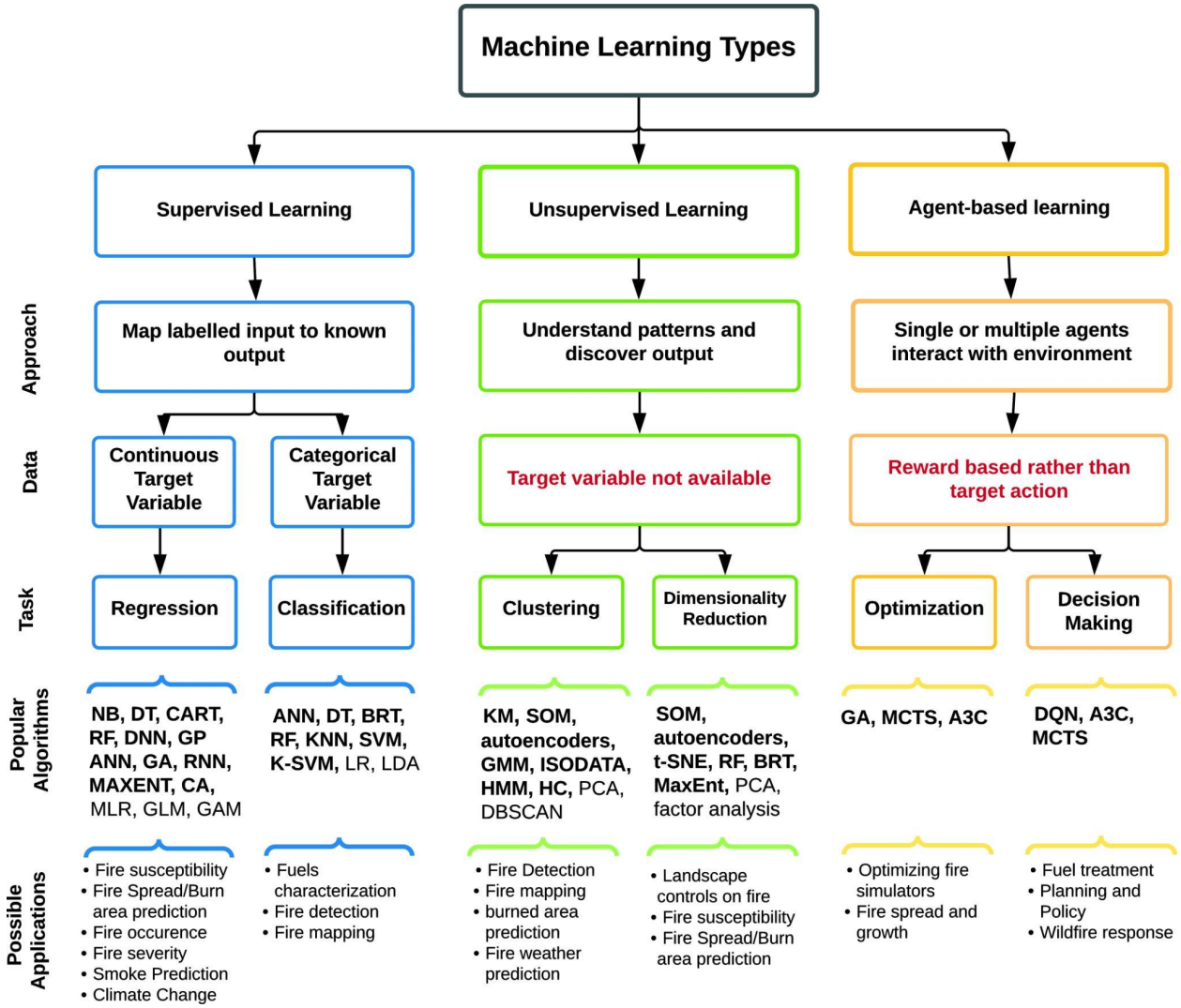
3 Related Work

There are not many papers that are very close to my project, mostly because my predictors are different from the papers online. However, here are three useful papers and projects that helped me wrap my head around what I did in case you would like to read further.

Mohajaneab et al. [5]: Data for this project was taken from NASA’s Fire Information for Resource Management System (FIRMS). The data was preprocessed by removing anomalous fires and out of season fires as most of Morocco’s fires occur during a dry season. Models were trained using a variety of factors including slope, elevation, aspect, landcover, NDVI, rainfall, temperature, wind, road distance, residential distance. This paper used a variety of machine learning models including RF-FR (AUC = 0.989), followed by SVM-FR (AUC = 0.959), MLP-FR (AUC = 0.858), CART-FR (AUC = 0.847), and LR-FR (AUC = 0.809). Obviously,

their favorite choice of metric was AUC. They did not specify other metrics. I would agree with them that RF-FR's metric is excellent and probably much better than any techniques we have learned so far for their data set.

Jain et al. [6]: This is a review paper with a very useful section (4.3.1) called Fire Occurrence Prediction which goes over a number of techniques and associated papers. While this project did not specifically perform any predictions it is still very useful, particularly for its suggestions of what models will work for our prediction problem. The following figure sums this information up nicely:



Kaggle Fire Prediction Project [7]: This project made use of the popular wildfire dataset on kaggle which contains data on 1.88 million wildfires. The author pre-processed the data by making sql queries to the database to get the relevant columns. The model was a simple neural network made with the pytorch framework. Curiously, they only reported the mean squared error loss for their model, and it was not very good. In fact, they report that it was only marginally

better than logistic regression. I would agree that compared to the more advanced models online that I have seen, this model is considerably weaker. At the same time, it uses techniques we have not even learned in class, and so I would probably do well to be able to out compete it, even if only just.

4 Methods

For each method, I will describe what took place and what if any effect it had on my analysis. For methods that are machine learning models, I will report my results in the hyper-parameter tuning phase used to verify a model and then report my results on the validation test set used to compare results between machine learning models after hyper-parameter tuning. Finally, I will transition to a discussion of more methods I experimented with but that need further analysis. The winning model was chosen based on the following criteria:

1) Are the fire and no fire predictions balanced? Here we define balanced as the 0-precision ("no-fire prediction") being over 0.75. If it is, proceed, otherwise, reject the model.
2) Is it the model with highest overall accuracy on the validation test set? If yes, select that model. If two are close, move on to the next criteria.
3) Which has the higher overall AUC-ROC? Choose that model.

These are by no means perfect criteria for selecting a model, but they work OK for our purposes because all of the models tend to over-predict yes relative to no, several models are very close in accuracy score, and these are close in AUC-ROC. As I previously stated, 0 refers to a prediction of no fire and 1 to a prediction of fire for a given state on a given day.

4.1 VIF Analysis

VIF Analysis was performed on all 32 parameters involved in the model. Significant multicollinearity was found. An output from the program showing the VIF scores of each parameter is available as item 1 in the appendix. Experimentation was performed to remove the multicollinear variables in steps, performing a new VIF analysis each time. Various models were attempted at each stage of the parameter removal. In no case did the model improve. Either an insignificant decrease or a large decrease in performance of the model occurred each time. In the end, most of this code was removed or hashed out in order to make the analysis less confusing. No parameters were removed via this method in the following analysis.

4.2 Train Test Split Without Randomization

As previously mentioned in the executive summary section, data were split into training, test, and validation sets. Parameter selection occurred on subsplits of the training set. Winning parameter selections were tested out on the validation set and trained on the training set. Models

were compared across their performance on the validation set and the winner was trained on the validation and training sets concatenated together (in order) and then tested on the test set. In this manner, the order of the training and test data was preserved in such a way that the future values were never used to predict the past values. This is why I call it "without randomization" as the usual package for train-test split via sci-kitlearn does introduce random orders, as do the methods used in the problem sets.

4.3 Multinomial Logistic Regression

4.3.a Hyper-Parameter Tuning

The default hyper parameters were used for the multinomial logistic regression model, as was done in the problem sets. No change in the parameters improved the model and therefore the default parameters were used. I will discuss what I would do in terms of hyper parameters in future work in the "future work" section. This produced an accuracy of about .7728 which was not very performant. Further, a classification report from sci-kitlearn produced the following results. These are a 0-precision of 0.69, a 0-recall of 0.61, a 1-precision of 0.81, and a 1-recall of 0.86. Clearly, there was not much need to investigate this model further.

4.3.b Testing on the Validation Set

No testing on the validation set was performed because the model was so poor. I simply left it alone and moved on to the next model.

4.4 XGBoost

4.4.a Hyper-Parameter Tuning

The default hyper parameters were used again for XGBoost. Any deviation from these produced worse results. Hyper parameter tuning for xgboost is mostly carried out by the algorithm anyway, so this is somewhat to be expected. Had I had more time, I would have investigated further, as mentioned in the future work section. For now, hyper parameter tuning produced a score of about 0.8438. Further a sci-kitlearn classification report led to the following: a 0-precision of 0.76, a 1-precision of 0.89, a 0-recall of 0.80, and a 1-recall of 0.87. Clearly, this was much more promising than logistic regression.

4.4.b Testing on the Validation Set

Testing on the validation set produced similar results with an accuracy score of about 0.8498. Further, for the validation sets, I will also report an AUC-ROC. In this case, the AUC-ROC was about 0.9246. Sci-kitlearn produced the following classification report results: a

0-precision of 0.75, a 1-precision of 0.91, a 0-recall of 0.83, and a 1-recall of 0.86.Clearly, we have a significant improvement over logistic regression and a solid start to our analysis.

4.5 Multilayer Perceptron

4.5.a Hyper-Parameter Tuning

Three structures for hyper parameter tuning were investigated in this project. First, a structure with one hidden layer and 40 neurons in that layer was investigated. Functions developed for the problem sets were employed to find the proper α values for the learning rate as well as the maximum iterations. This was found to always be the lowest learning rate available and therefore α was set to be .01, as this was the lowest learning rate I tested. The activation parameter was set to "logistic," the solver was set to "adam," and max_iter was set to 30 (determined by the aforementioned functions). All other parameters were set to their defaults. I will refer to these as "standard parameters" from here on out. This model produced a score of about 0.8349 and the following classification report results: a 0-precision of 0.71, a 1-precision of 0.93, a 0-recall of 0.88, and a 1-recall of 0.81. This is less strong of a result compared to XGBoost, but still promising. Interestingly, the precision for "1" has increased whereas the precision for "0" decreased. This is probably indicative of the model over predicting "1" (fire) as opposed to "0" (no fire).

The next model contained 2 hidden layers with 40 neurons in each layer. Functions developed for the problem sets were again used to find the proper α values for the learning rate as well as the maximum number of iterations. All other parameters were set to "standard," as was previously explained. Again, an α value of .01 was chosen to be the best, probably because this was the lowest value tested. This produced a score of about 0.8269, a 0-precision of .69 a 0-recall of .90, a 1-precision of .94, and a 1-recall of about .79. This is not an improvement over the previous model and so we will transition to discussing the third model.

The final multi-layer perceptron model I tested was also set to standard parameters because this was also what the functions developed for the problem sets decided was best. It contained 3 hidden layers of 40 neurons each. It produced an accuracy score of 0.8363, a 0-precision of 0.76, a 0-recall of .77, a 1-precision of .88, and a 1-recall of .87. This is probably best described as a slight improvement over the other two models, however the 1-precision is still lower than XGBoost. Given more time, I would have investigated more structures with more numbers of neurons on each hidden layer. This is covered in the future work section.

4.5.b Testing on the Validation Set

The most performant model to come out of hyper-parameter tuning was the three-layer model of 40 neurons each and so this was trained on the training set and tested on the validation set. Once again, all parameters were set to "standard." As before, with testing on the validation set, I also included an AUC-ROC score. For some reason, this model produced an AUC-ROC of

about 0.1848, which indicates that it is probably not worth investigating further. Strangely, the accuracy score was excellent compared to what we have seen so far, about 0.8432. The 0-precision and 0-recall were 0.73 and 0.85 respectively and the 1-precision and 1-recall were 0.92 and 0.84 respectively.

4.6 K-Nearest Neighbors

4.6.a Hyper-Parameter Tuning

For the k-nearest neighbors algorithm, the only parameter I adjusted for my analysis was the number of neighbors ("n_neighbors"). I used a for-loop to iterate through fitting the model with each number of neighbors and then compared them to decide which produced the lowest root-mean-squared error (RMSE). Data were scaled before this process using the StandardScalar() module from sci-kitlearn. According to this analysis, the number of neighbors was chosen to be 25. This produced an RMSE of about 0.4034 which is not stellar considering our scaling. However, it is on par with our previous results.

4.6.b Testing on the Validation Set

Using the model with 25 neighbors, I again fit with the training data and tested on the validation data. This produced an accuracy score of about 0.8421 and an AUC-ROC of about 0.7155, which is an improvement over our last model, but still a far cry from XGBoost. We have a 0-precision of 0.73, a 0-recall of 0.84, a 1-precision of 0.91, and a 1-recall of 0.85. For some reason, this again translated to a poor AUC-ROC score.

4.7 Random Forest Classifier

4.7.a Hyper-Parameter Tuning

Despite trying a plethora of hyper parameters, no improvement was found over the default hyper-parameters for the random forest classifier. This makes sense as these algorithms are essentially implemented as smart according to the documentation. Parameter tuning on the training set produced the following results. We have a 0-precision of 0.75, a 0-recall of 0.82, a 1-precision of 0.90, and a 1-recall of 0.86. Further, we have an overall accuracy of about 0.8452. This is very promising and probably the best result so far, depending on the testing on the validation set.

4.7.b Testing on the Validation Set

This same model was trained on the training data and tested on the validation data with the following results. We have a 0-precision of 0.75 and a 0-recall of 0.85 with a 1-precision 0f

0.92 and a 1-recall of 0.86. We have an overall score of about 0.8550 with an AUC-ROC of 0.9252. These results barely edge out our XGBoost classifier model.

4.8 Logistic GAM Classifier

4.8.a Hyper-Parameter Tuning

A logistic GAM model was fit to the training data, again with default parameters, this time because we are using so many parameters for fitting (26) that any attempt to systematically adjust hyper parameters resulted in killing the kernel. Further, less systematic attempts did not result in any improvement whatsoever and so it is unlikely that a more systematic approach would have yielded any better results. Hyper-parameter tuning yielded the following results. Interestingly, these are by far the best results so far, especially in terms of the 0-precision. However, as we will see, logisticGAM is inconsistent. I will explore this in further detail in the future work section. Our 0-precision was 0.79 with a 0-recall of 0.80 and 1-precision and 1-recall of 0.9 and 0.89 respectively. Overall, our accuracy score was about 0.8620.

4.8.b Testing on the Validation Set

This same model was again fit to the training data and tested on the validation data. This resulted in a score of about 0.8420 and an AUC-ROC of about 0.9225. The rest of our results are as follows. These are a 0-precision of 0.74, a 0-recall of 0.81, a 1-precision of 0.90, and a 1-recall of 0.86. For some reason, and this will come out in more detail later, logistic GAM is high performance but inconsistent. For some arbitrary time periods, it will spike so that the 0-precision is in the high .80s, but for the same interval of time over a different period, it could be as low as .40. This obviously deserves further investigation, and I will discuss it further in the following sections.

4.9 Tree Classifier

4.9.a Hyper-Parameter Tuning

A tree-based classifier model was fit to the data with default parameters except for max_depth being set equal to 7. It was determined that this was the best parameter combination for the training data set, and this led to the following results. These are an accuracy score of about 0.8228, a 0-precision of .73, a 0-recall of .75, a 1-precision of .87, and a 1-recall of .86. After testing this model on the training data set, I transitioned to testing on the validation data set.

4.9.b Testing on the Validation Set

Our tree-based classifier with max-depth = 7 yielded the following results on the validation data set. Our accuracy score for this model was about 0.8330 with an AUC-ROC of about 0.9097. I report a 0-precision and 0-recall of 0.72 and 0.81 respectively along with a 1-precision and 1-recall of 0.90 and 0.85 respectively. Obviously, this is a decent classifier, but it does not compare favorably to our random forest model.

4.10 Support Vector Classifier

4.10.a Hyper-Parameter Tuning

Hyper-parameter tuning for the support vector classifier took approximately 1.5 Hours due to the necessity of an extensive hyper-parameter search for this model. A grid search was performed over the following parameters: C - 0.1, 1, 10, 100, 1000; gamma - 1, 0.1, 0.01, 0.001, 0.0001; kernel - 'rbf'. This took so long that I only had time to analyze the rbf kernel. I have no reason to suspect that other kernels would perform better as this model was on par with all of the other models. The best parameters were chosen as C=100, gamma=.0001, and kernel="rbf". Further, probability was set to True in order to allow for an AUC-ROC calculation later. This yielded the following results. These are a 0-precision of 0.73, a 0-recall of 0.80, a 1-precision of 0.89, and a 1-recall of 0.85. Finally, I report an accuracy score of about 0.8323. This is clearly a promising model and so we move to test on the validation set.

4.10.b Testing on the Validation Set

After testing on the validation set, I report the following results.These are an accuracy score of 0.8389 and an AUC-ROC of 0.8952. Clearly, this is a decent classifier but it is not on par with XGBoost or Random Forest (RF). I report a 0-precision of 0.73 a 0-recall of 0.82, a 1-precision of 0.90, and a 1-recall of 0.85.

4.11 Naive Bayes Classifier

4.11.a Hyper-Parameter Tuning

Hyper-parameter tuning was carried out across various values of alpha ranging from 0.01 to 10. The winning model generated an accuracy score of 0.8085, a 0-precision of 0.71, a 0-recall of 0.74, a 1-precision of 0.86, and a 1-recall of 0.85. This is not very strong relative to our other models; however, I still tested on the validation set in case things improved.

4.11.bTesting on the Validation Set

The winning model from the hyper-parameter tuning stage was fit to the training data and tested on the validation set. This yielded the following results. These are a 0-precision of 0.70, a 0-recall of 0.76, a 1-precision of 0.87, and a 1-recall of .84. Further, our model generated an accuracy score of 0.8131, and an AUC-ROC of 0.8836.

4.12 XGBoost Random Forest Ensemble Methods

4.12.a Hyper-Parameter Tuning

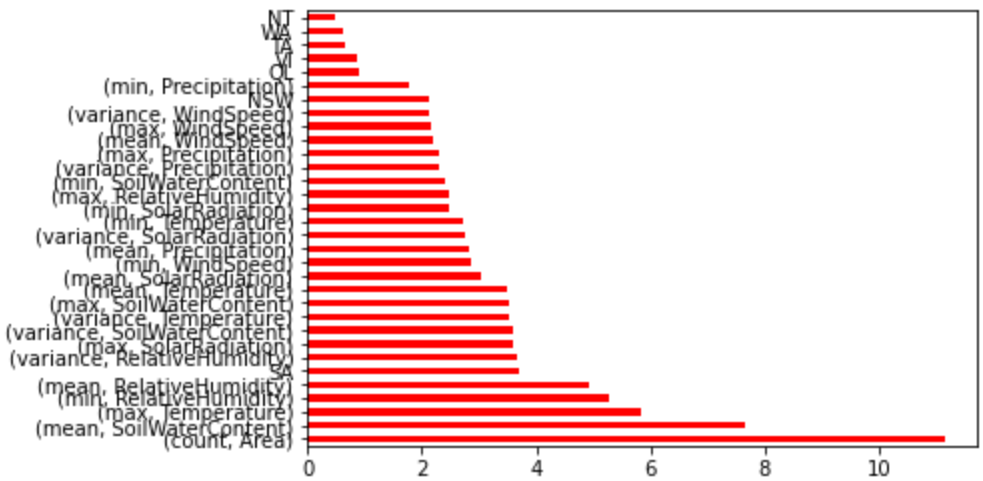
Hyper-parameter tuning for this ensemble method occurred over the max_depth hyper-parameter. All others were set to default. The optimal max-depth on the training data was found to be 15. This model yielded the following results. These are a 0-precision of 0.75, a 0-recall of 0.82, a 1-precision of 0.90, and a 1-recall of 0.86. Further, the model generated an accuracy score of approximately 0.8472. As you might have imagined, these are on par with the XGBoost and Random Forest models.

4.12.b Testing on the Validation Set

This model was then trained on the training set and tested on the validation set. This led to the following results. These are a 0-precision of 0.74, a 0-recall of 0.83, a 1-precision of 0.91, a 1-recall of 0.86, an overall accuracy of about 0.8490, and an AUC-ROC score of 0.9253.

4.13 Experimentation with Bagging and Importance

Because bagging can also generate a metric for the importance of parameters, under the advice of Professor Sahota, I selected the top 5 most important parameters and repeated some of the models mentioned above. In each case, the performance was worse instead of better. Since I am primarily attempting this project as a proof of concept, I focused on trying to get the highest scores possible rather than attempting to narrow down the minimal number of parameters. It is certainly true that similar models could be generated with a fraction of the 26 parameters I used.



In no case was I able to find a model that performed better than my models in terms of the criteria mentioned under “methods,” using a subset of the 26 predictors. Although, I did attempt this using bagging and importance. With more time, it may have been possible to find one, and I will discuss this further in the future direction section.

4.14 Experimentation with LogisticGAM

Because logisticGAM was somehow able to generate a 0-precision of 0.79, the best of any model on the training data, I experimented with running logisticGAM over various time periods in my dataset. For example, with time periods of a few months to a few days for both training and testing. Preliminarily, what I found was that this result was a fluke and that logisticGAM models were inconsistent for this data set. An open question for further analysis is whether with a subset of predictors or with better multicollinearity adjustments, it might be possible to find a better logisticGAM model. It may also be that it works for time series of one state alone. I did not have time to delve into either of these two questions but I will discuss them in the future work section.

4.15 Experimentation with Resampling Methods

I will now discuss what I call resampling methods. There is a slight imbalance in my data of 2:1 fire to no-fire which is an artifact of the granularity ,or rather the lack thereof, in my data because I am aggregating weather conditions over entire states in Australia. Even though it would violate the condition that we not use the future to predict the past, I wanted to see what effect, if any, randomly sampling the dataset so that there was a 1:1 fire to no-fire ratio would have. It turns out that the 0-precision went down whereas the 1-precision went up. So the accuracy stayed roughly the same across models, except the precision became more “imbalanced”. Why this would be the case is yet another open question for my future work section.

4.16 Data Table Pivot

I would be remiss if I did not mention one simple yet powerful technique I used to analyze this data set, and that is pivoting a dataframe. At first this might seem simple, but it is deceptively difficult to explain, and so I will describe this work of magic by the pandas developers as a semi-transpose of the data and then include screenshots to show what I mean. Hopefully, this helps someone else in the class. Here we have the original data frame:

Date	Region	Parameter	count()[unit: km^2]	min()	max()	mean()	variance()
2005-01-01	NSW	Precipitation	800234.348986	0.000000	1.836935	0.044274	0.028362
2005-01-01	NSW	RelativeHumidity	800234.348986	13.877194	80.522964	36.355567	253.559937
2005-01-01	NSW	SoilWaterContent	800234.348986	0.002245	0.414305	0.170931	0.007758
2005-01-01	NSW	SolarRadiation	800234.348986	14.515009	32.169781	26.749389	6.078587
2005-01-01	NSW	Temperature	800234.348986	14.485785	35.878704	27.341182	18.562212

As you can see, we have parameters for each state organized in a way that makes it impossible to plug into any of the models we covered in class. Now, we have the pivoted data frame:

		NSW	NT	QL	SA	TA	VI	WA	(count, Area)	(min, Precipitation)	(mi RelativeHumidit
Date	**Region**										
2005-01-01	**NSW**	1	0	0	0	0	0	0	8.002343e+05	0.000000	13.8771
	NT	0	1	0	0	0	0	0	1.357561e+06	0.000000	14.5588
	QL	0	0	1	0	0	0	0	1.730605e+06	0.000000	14.4431
	SA	0	0	0	1	0	0	0	9.913151e+05	0.000000	10.6181
	TA	0	0	0	0	1	0	0	6.778526e+04	0.003973	43.9065
...	**...**	...	...	...	...	...	...	...	...	...	
2020-10-31	**QL**	0	0	1	0	0	0	0	1.730605e+06	0.000000	12.7499
	SA	0	0	0	1	0	0	0	9.913151e+05	0.000000	12.5123
	TA	0	0	0	0	1	0	0	6.778526e+04	0.000703	69.8010
	VI	0	0	0	0	0	1	0	2.294532e+05	0.000000	66.4592
	WA	0	0	0	0	0	0	1	2.528546e+06	0.000000	9.2727

with one-hot encoded states and pandas multi-indices for each parameter. The code to do this in python3/pandas (less the one-hot encoding) is simply:

```
df =df.pivot(index=['Date']+['Region'], columns=['Parameter'])
```

which is worth mentioning in this report in case it saves even one person from attempting to manipulate data without using pandas.

5 Future Directions

This section is for open questions which I still have after completing my analysis.

1. Are there better hyper-parameters for the models where I used the package's default hyperparameters. Most of these I experimented with, but I did not use gridsearch for all of them because it took so long to run and because it seemed unnecessary given preliminary tweaking by hand producing no change.
2. Is there some configuration or use of logisticGAM models which I did not anticipate which can produce better results than random forest consistently. I would start by attempting to separate the data by state and then try logisticGAM for each state. I would also try even more configurations of training time periods and testing time periods. Perhaps there is a magic ratio of lead time to prediction time that I missed. This could be carried out systematically, but it would take a lot of time to fit all of those models. LogisticGAM is not particularly fast.
3. Finally, is there some subset of predictors that can be used that give just as good a model on average? I am fairly certain that there is no subset that would be markedly better, but I have not investigated enough to be sure that I could not produce approximately the same results with 5 predictors for example.
4. There is a plethora of satellite imagery and vegetation index data in the original dataset. Would using any of this data improve our model?

6 Description of Effort

6.1 What I Learned vs. What I Knew

I possessed most of the skills I needed to predict forest fires before starting this project. All I really had to do was spend time on learning specific features of sci-kitlearn as well as its syntax. I also had to figure out how to pivot a table which took several hours. The rest was more than covered in class.

6.2 Libraries and Packages Used

I did not write any of my own packages to analyze this data set. I mostly used sci-kitlearn to leverage pre-built, supervised machine learning models and pandas for manipulating data. For a full list of specific imports please see my github submission. Imports are listed at the very top. I used the following modules: pandas, datetime, re, sklearn, xgboost, numpy matplotlib, tqdm, pygam, and statsmodels.

6.3 Allocation of Time

The vast majority of my time was spent looking for a dataset that was OK and building the models for the data set. What I mean by a good data set is that it only required a reasonable amount of cleaning and that it was appropriate for models from this class. This dataset happens to be particularly strong in that you could incorporate data about the vegetation and satellite imagery into any analysis. You could even do regression on the intensity of the fires along with many other such analyses. I chose classification because it would have probably been beyond the scope of the class to do anything else, but the data is more than adequate for more advanced models. The goal of the competition the data was taken from was predicting facts about future wildfires using historical data in combination with weather forecasts and their associated lead times. One team of professionals from IBM reportedly spent 150 hours analyzing this data set, and so I felt it would be more appropriate to build a simpler model. Another thing I liked about this data set as opposed to a Kaggle dataset, is that there were no answers published for this data set that I am aware of. I was truly on my own to fail or succeed. The rest of my time was spent writing this paper and preparing for the presentation. This took considerably less time.

7 Appendix

Item 1:

	feature	VIF
0	(min, WindSpeed)	4.977948e+00
1	NSW	inf
2	(mean, Temperature)	4.457037e+01
3	(max, RelativeHumidity)	2.960226e+00
4	(mean, WindSpeed)	7.356593e+00
5	(min, SoilWaterContent)	2.623690e+00
6	(min, SolarRadiation)	7.710638e+00
7	WA	inf
8	fire01	1.921411e+00
9	(min, RelativeHumidity)	2.106018e+01
10	QL	inf
11	(variance, Precipitation)	3.603813e+00
12	(mean, SoilWaterContent)	1.571319e+01
13	(mean, SolarRadiation)	3.219168e+01
14	(mean, RelativeHumidity)	1.946206e+01
15	(count, Area)	inf
16	(variance, Temperature)	5.731700e+00
17	(max, Precipitation)	4.052888e+00
18	VI	inf
19	(variance, WindSpeed)	3.633465e+00
20	SA	inf
21	TA	9.007199e+15
22	NT	inf
23	(min, Precipitation)	1.611378e+00
24	(max, Temperature)	4.754164e+01
25	(variance, SoilWaterContent)	5.015106e+00
26	(variance, SolarRadiation)	4.974307e+00
27	(max, SoilWaterContent)	3.457697e+00
28	(max, WindSpeed)	2.886077e+00
29	(mean, Precipitation)	4.550328e+00
30	(variance, RelativeHumidity)	5.536556e+00
31	(min, Temperature)	2.192228e+01
32	(max, SolarRadiation)	2.885985e+01

8 Bibliography

[1] J. S. Hendrik Hamann, “Predicting wildfires with weather forecast data,” 2021. https://developer.ibm.com/exchanges/data/all/spot-challenge-wildfires/.

[2] M. Foley, “The high cost of wildfire in 2018,” 2018. https://www.nfpa.org/News-and-Research/Publications-and-media/NFPA-Journal/2019/November-December-2019/Features/Large-Loss/Wildfire-Sidebar: :text=Residential

[3] S. Reinberg, “Wildfires cause more than 33,000 deaths globally each year,” 2021. https://www.usnews.com/news/health-news/articles/2021-09-09/wildfires-cause-more-than-33-000-deaths-globally-each-year.

[4] V. Gill, “Brazil wildfires killed an estimated 17 million animals,” 2021. https://www.bbc.com/news/science-environment-59670396.

[5] F. K. Q. B. P. A. E. H. N. G. L. F. O. Meriame Mohajaneab, Romulus Costachec, “Application of remote sensing and machine learning algorithms for forest fire mapping in a mediterranean area,” 2018. https://www.sciencedirect.com/science/article/pii/S1470160X21005343?via

[6] S. G. S. M. C. S. T. M. D. F. Piyush Jain, Sean C.P. Coogan, “A review of machine learning applications in wildfire science and management,” 2020. https://cdnsciencepub.com/doi/full/10.1139/er-2020-0019.

[7] A. S. M. L. to predict wildfires using PyTorch Lightning, “Leveraging machine learning to predict wildfires using pytorch lightning,” 2020. https://towardsdatascience.com/leveraging-machine-learning-to-predict-wildfires-contributing-to-the-united-nations-sustainable-a10c5044dcae